Timestamp Event Service Design

Interview-ready summary

Proposed production architecture:

Client -> ALB -> ECS Fargate tasks (AZ-a + AZ-b) -> RDS PostgreSQL Multi-AZ primary/standby -> async cross-region replica.

1. Implemented design

This repository implements one Flask + Gunicorn application, one PostgreSQL database, Docker Compose for local deployment, a named database volume, validation, committed inserts, and health endpoints.

2. Request flow

Client -> Flask app -> PostgreSQL insert -> commit -> return saved record. The app validates the request, inserts the row, and closes the connection after the request finishes.

3. Proposed production architecture

Client -> ALB -> ECS Fargate tasks in AZ-a and AZ-b -> RDS PostgreSQL Multi-AZ primary/standby (region 1) -> async cross-region replica (region 2).

4. Availability and redundancy

Run app tasks across at least two availability zones. Keep an Application Load Balancer in front of the apps and a PostgreSQL standby in another zone. Describe recovery time and possible data loss explicitly.

5. Elasticity

Propose an initial autoscaling policy such as minimum 2 tasks, maximum 10 tasks, and CPU target 60%. These values need load testing and are not implemented in Docker Compose.

6. Automation gaps

Local automation is implemented in Docker Compose. Cloud deployment, autoscaling, migration automation, backup restoration, and regional recovery remain proposed follow-up work.